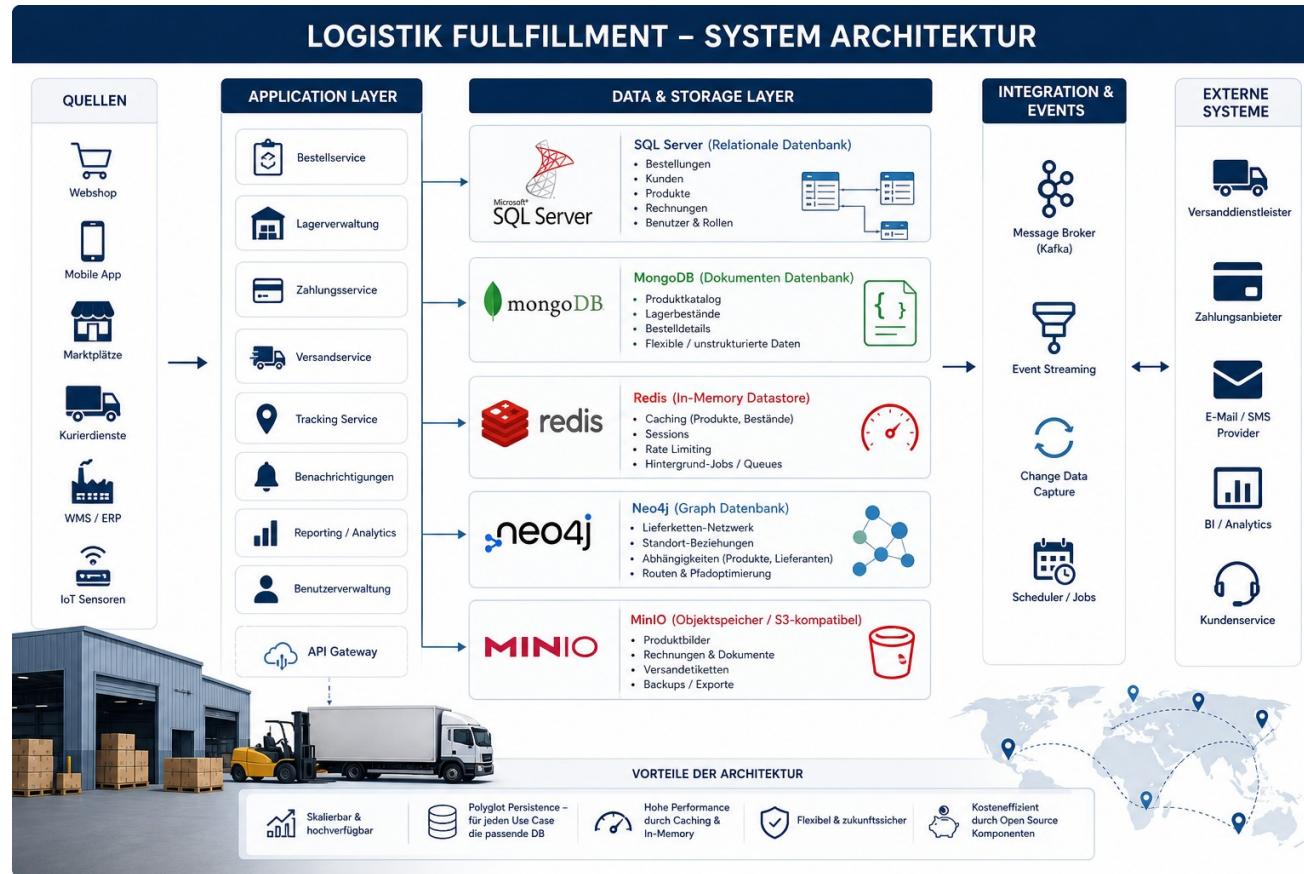


Manual Datenmanagement

Logistikbeispiel



Aufgabe:

- führen Sie jeden Schritt aus dem manual auf Ihrem Computer aus (der git-Teil erfolgt einmal pro Team/repository, der restliche Teil bei jedem lokal auf dem Computer)
- Verschaffen Sie sich dabei einen Überblick über die Datenmodelle und die Funktionsweise der Datenbanken und der python Skripte. Beschreiben Sie was konkret in den python Skripten passiert.
- Entwerfen Sie für das Relationale Datenmodell ein ER-Modell

Schritt 1: .zip file mit allen Projektdateien aus Moodle-Lernraum herunterladen

Eine Person aus Ihrem Team lädt das .zip mit der folder-Struktur herunter

✓ **Vorlage_und_Manual_Logistik_Beispiel_Datenbanken** ✎



datasemodels_logistics_playground ✎ ZIP



2.1 MB

+ Aktivität oder Material anlegen

Schritt 2: Ordnerstruktur und files manuell / über mylab gitlab website in Gruppen repo Projekt anlegen

Eine Person aus Ihrem Team geht in Ihr git-Team repo auf der mylab gitlab website und legt das Verzeichnis: "playground_logistics_databasemodels" an

The screenshot shows the GitLab web interface for a group repository named "gruppe10_dma_sose26". The browser address bar shows the URL "git.mylab.th-luebeck.de/teaching/drews/gruppe10_dma_sose26". The interface includes a sidebar with navigation options like "Pinned", "Merge requests", "Work items", "Labels", "Manage", "Plan", "Code", "Build", "Secure", "Deploy", "Operate", "Monitor", "Analyze", and "Settings". The main content area displays the repository details, including the "Initial commit" by "Drews, André" 3 weeks ago. A table lists the files in the repository, showing "README.md" as the only file. A dropdown menu is open, showing options to "New file", "Upload file", "New directory", "New branch", and "New tag". The "Project information" section on the right shows "1 Commit", "1 Branch", "0 Tags", and "4 KiB Project Storage". The "README.md" file content is displayed, including a "Getting started" section and a "Add your files" section with instructions on how to create or upload files.

Project: **gruppe10_dma_sose26**

Lehre / Drews / **gruppe10_dma_sose26**

gruppe10_dma_sose26

main gruppe10_dma_sose26

Initial commit
Drews, André authored 3 weeks ago

Name	Last commit
README.md	Initial commit

README.md

gruppe10_dma_sose26

Getting started

To make it easy for you to get started with GitLab, here's a list of recommended next steps.

Already a pro? Just edit this README.md and make it your own. Want to make it easy? [Use the template at the bottom!](#)

Add your files

- Create or upload files
- Add files using the command line or push an existing Git repository with the following command:

```
cd existing_repo
git remote add origin https://git.mylab.th-luebeck.de/teaching/drews/gruppe10_dma_sose26.git
git branch -M main
git push -uf origin main
```

Project information

- 1 Commit
- 1 Branch
- 0 Tags
- 4 KiB Project Storage

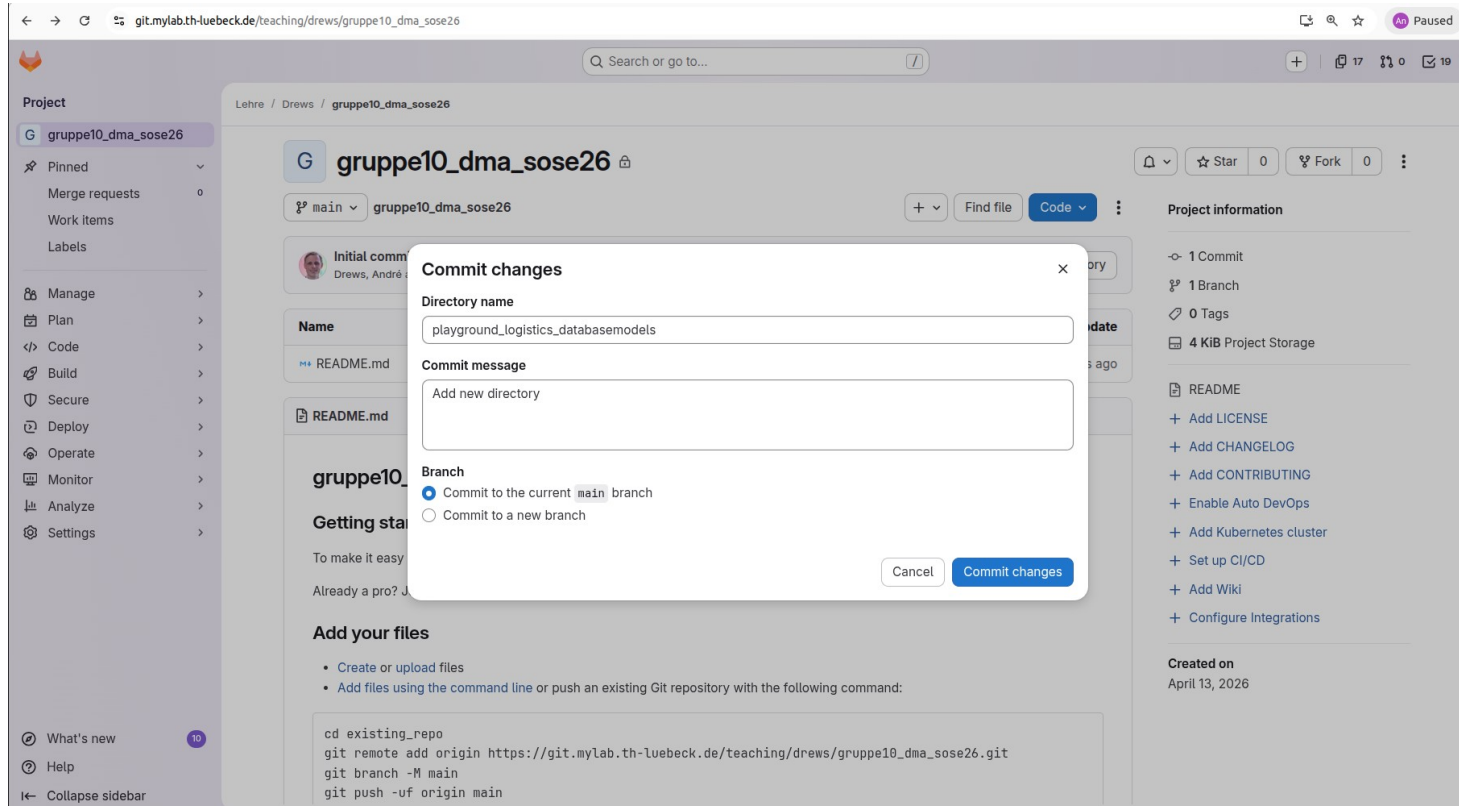
README

- + Add LICENSE
- + Add CHANGELOG
- + Add CONTRIBUTING
- + Enable Auto DevOps
- + Add Kubernetes cluster
- + Set up CI/CD
- + Add Wiki
- + Configure Integrations

Created on
April 13, 2026

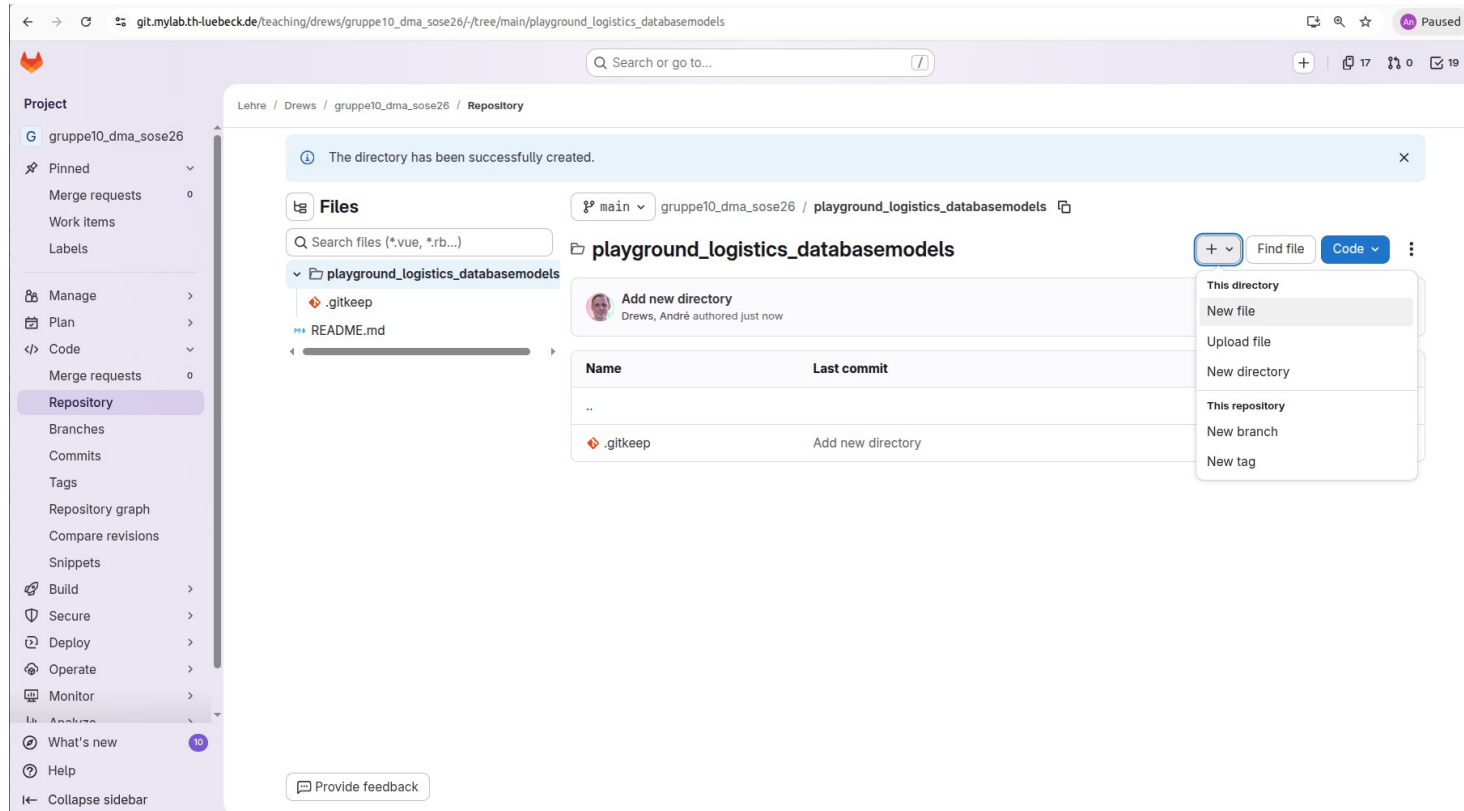
Schritt 2: Ordnerstruktur und files manuell / über mylab gitlab website in Gruppen repo Projekt anlegen

Eine Person aus Ihrem Team geht in Ihr git-Team repo auf der mylab gitlab website und legt das Verzeichnis: "playground_logistics_databasemodels" an



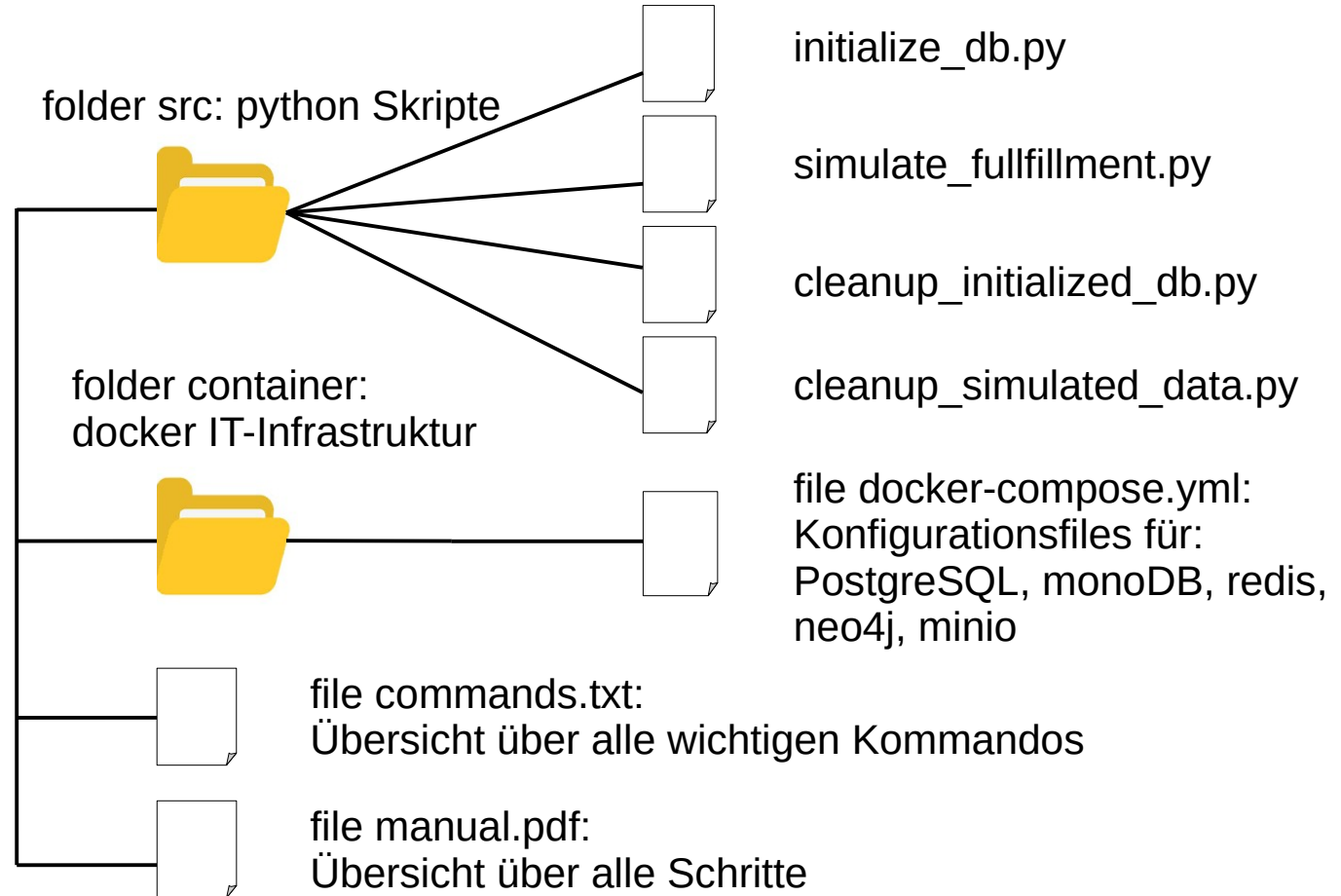
Schritt 2: Ordnerstruktur und files manuell / über mylab gitlab gitlab-website in Gruppen repo Projekt anlegen

Eine Person aus Ihrem Team geht in Ihr git-Team repo auf der mylab gitlab, klickt in das neu angelegte Verzeichnis "playground_logistics_databasemodels"



Schritt 2: Ordnerstruktur und files manuell / über mylab gitlab-website in Gruppen repo Projekt anlegen

folder-Struktur:



Schritt 2: Ordnerstruktur und files manuell / über mylab gitlab gitlab-website in Gruppen repo Projekt anlegen

Eine Person aus Ihrem Team geht in Ihr git-Team repo auf der mylab gitlab und legt die Datenstruktur (Ordner "src", "container und Dateien) im repo manuell an

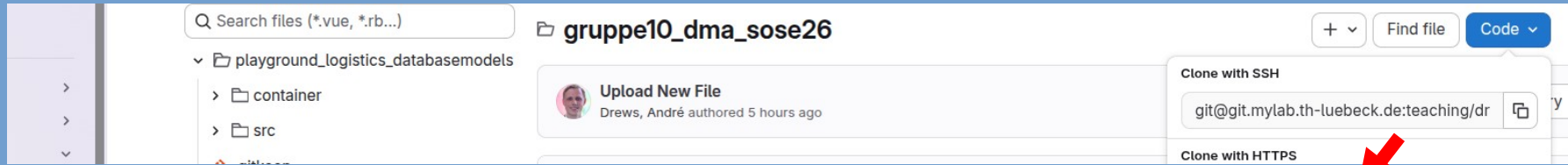
The screenshot shows the GitLab web interface for a repository named 'gruppe10_dma_rose26'. The left sidebar contains a 'Project' menu with options like 'Pinned', 'Merge requests', 'Work items', 'Labels', 'Manage', 'Plan', 'Code', 'Merge requests', 'Repository' (selected), 'Branches', 'Commits', 'Tags', 'Repository graph', 'Compare revisions', 'Snippets', 'Build', and 'Secure'. The main content area shows the 'Repository' view for the path 'Lehre / Drews / gruppe10_dma_rose26 / Repository'. A blue notification banner at the top states 'The directory has been successfully created.' Below this, the 'Files' section shows a tree view of the repository. The 'playground_logistics_databasemodels' directory is expanded, showing subdirectories 'container' and 'src', and files '.gitkeep', 'commands.txt', 'manual.odp', and 'README.md'. The right sidebar shows the 'playground_logistics_databasemodels' directory with a '+', 'Find file', and 'Code' button. Below this is an 'Upload New File' section with a user profile and the text 'Drews, André authored 5 hours ago'. A table lists the files and directories in the repository, showing their names, last commit, and last update time.

Name	Last commit	Last update
..		
container	Add new directory	5 hours ago
src	Upload New File	5 hours ago
.gitkeep	Add new directory	16 hours ago
commands.txt	Upload New File	5 hours ago
manual.pdf	Upload New File	5 hours ago

Schritt 2: repo Neuerungen in lokales Verzeichnis "pullen"

Jeder! aus Ihrem Team geht in sein lokales Vorlesungsverzeichnis in das er mal geclonet hat, und gibt den `git status` und danach `git pull` Befehl ein.

gecloned sollten Sie bereits haben (diesen Schritt müssen Sie in der Regel nicht nochmal machen. Falls gar nichts funktioniert, können Sie nochmal in ein neues Verzeichnis clonen)



```
ad@ad-ThinkPad-T460s:~/Desktop/1_dma/Aufgaben/VL_dma$ git clone git@git.mylab.th-luebeck.de:
Cloning into 'gruppe10_dma_ose26'...
remote: Enumerating objects: 29, done.
remote: Counting objects: 100% (29/29), done.
remote: Compressing objects: 100% (26/26), done.
remote: Total 29 (delta 9), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (29/29), 2.08 MiB | 7.39 MiB/s, done.
Resolving deltas: 100% (9/9), done.
```

Hier steht Ihre URL

git

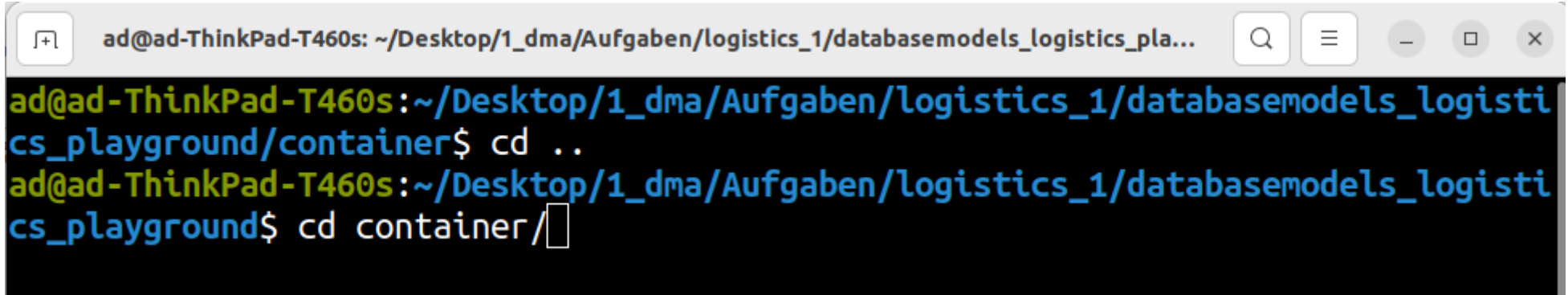
```
ad@ad-ThinkPad-T460s:~/Desktop/1_dma/Aufgaben/VL_dma/gruppe10_dma_ose26$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
ad@ad-ThinkPad-T460s:~/Desktop/1_dma/Aufgaben/VL_dma/gruppe10_dma_ose26$ git pull
Already up to date.
```

Diese Befehle muss jeder ausführen!

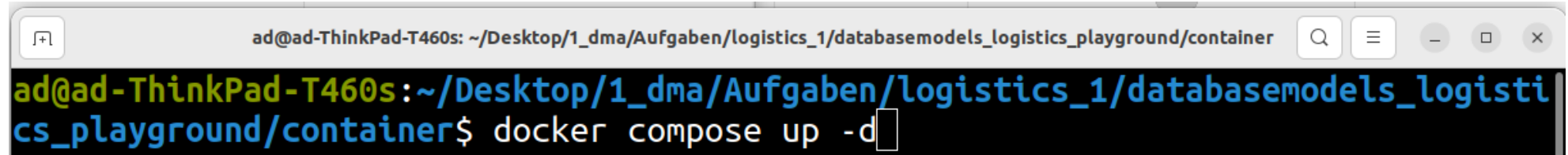
Schritt 3: IT-Infrastruktur starten

Jeder wechselt in das Verzeichnis container in dem das Konfigurations file "docker-compose.yml" liegt

A terminal window with a title bar showing the path ~/Desktop/1_dma/Aufgaben/logistics_1/databasemodels_logistics_pla... The terminal text shows a user named 'ad' at a 'ThinkPad-T460s' machine. The prompt is ~/Desktop/1_dma/Aufgaben/logistics_1/databasemodels_logistics_playground/container\$. The user enters 'cd ..' and the prompt changes to ~/Desktop/1_dma/Aufgaben/logistics_1/databasemodels_logistics_playground\$. Then the user enters 'cd container/' and the prompt changes to ~/Desktop/1_dma/Aufgaben/logistics_1/databasemodels_logistics_playground/container\$.

```
ad@ad-ThinkPad-T460s: ~/Desktop/1_dma/Aufgaben/logistics_1/databasemodels_logistics_pla...
ad@ad-ThinkPad-T460s:~/Desktop/1_dma/Aufgaben/logistics_1/databasemodels_logistics_playground/container$ cd ..
ad@ad-ThinkPad-T460s:~/Desktop/1_dma/Aufgaben/logistics_1/databasemodels_logistics_playground$ cd container/
```

docker compose up -d ausführen und damit Datenbanken/container starten

A terminal window with a title bar showing the path ~/Desktop/1_dma/Aufgaben/logistics_1/databasemodels_logistics_playground/container. The terminal text shows the user 'ad' at 'ThinkPad-T460s' with the prompt ~/Desktop/1_dma/Aufgaben/logistics_1/databasemodels_logistics_playground/container\$. The user enters the command 'docker compose up -d' and the cursor is at the end of the command.

```
ad@ad-ThinkPad-T460s: ~/Desktop/1_dma/Aufgaben/logistics_1/databasemodels_logistics_playground/container
ad@ad-ThinkPad-T460s:~/Desktop/1_dma/Aufgaben/logistics_1/databasemodels_logistics_playground/container$ docker compose up -d
```

Schritt 3: IT-Infrastruktur starten

Ausgabe:

```
ad@ad-ThinkPad-T460s: ~/Desktop/1_dma/Aufgaben/logistics_1/databasemodels_logistics_playground/container
ad@ad-ThinkPad-T460s:~/Desktop/1_dma/Aufgaben/logistics_1/databasemodels_logistics_playground/container$ docker compose up -d
WARN[0000] /home/ad/Desktop/1_dma/Aufgaben/logistics_1/databasemodels_logistics_playground/container/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] up 6/6
✓ Container redis      Started      3.0s
✓ Container minio      Started      4.2s
✓ Container postgres   Started      2.4s
✓ Container neo4j       Started      3.8s
✓ Container mongodb    Started      2.8s
✓ Container cleanup     Started      1.9s
```

Schritt 3: Überprüfung laufende container

Mit `docker ps` überprüfen welche Datenbanken/container laufen:

```
ad@ad-ThinkPad-T460s:~/Desktop/1_dma/Aufgaben/logistics_1/databasemodels_logistics_playground/container$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
3c96696f5897	postgres:15	"bash -c ' while tru..."	26 hours ago	Up 2 minutes	5432/tcp	cleanup
ad1c704f2996	minio/minio	"/usr/bin/docker-ent..."	26 hours ago	Up 2 minutes	0.0.0.0:9000-9001->9000-9001/tcp, [::]:9000-9001->9000-9001/tcp	minio
110ff8f6f8b7	redis:7	"docker-entrypoint.s..."	26 hours ago	Up 2 minutes	0.0.0.0:6379->6379/tcp, [::]:6379->6379/tcp	redis
88129757fa9d	postgres:15	"docker-entrypoint.s..."	26 hours ago	Up 2 minutes	0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp	postgres
b7ede0bdffc8	mongo:7	"docker-entrypoint.s..."	26 hours ago	Up 2 minutes	0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp	mongodb
71f2467941f5	neo4j:5	"tini -g -- /startup..."	26 hours ago	Up 2 minutes	0.0.0.0:7474->7474/tcp, [::]:7474->7474/tcp, 7473/tcp, 0.0.0.0:7687->7687/tcp, [::]:7687->7687/tcp	neo4j

Schritt 4: in postgresQL container

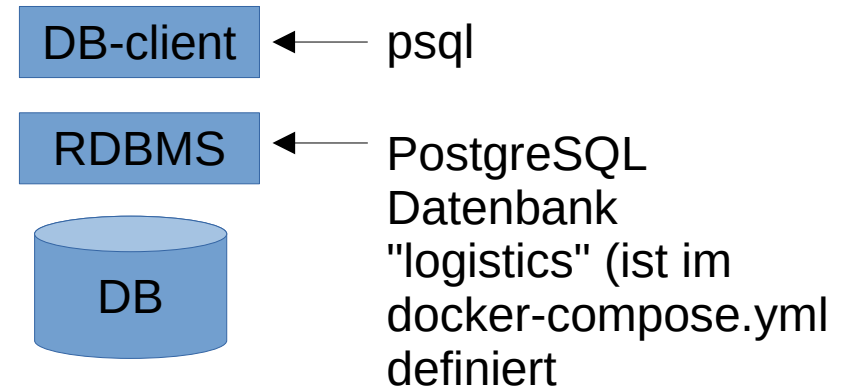
Über `docker exec -it postgres /bin/bash` wird in den postgresDB-container gesprungen:

```
ad@ad-ThinkPad-T460s:~/Desktop/1_dma/Aufgaben/logistics_1/databasemodels_logistics_playground/container$ docker exec -it postgres /bin/bash
root@88129757fa9d:/#
```

Über `psql -U user -d logistics` wird über den SQL-client `psql` auf die Datenbank "logistics" zugegriffen:

```
root@88129757fa9d:/# psql -U user -d logistics
psql (15.17 (Debian 15.17-1.pgdg13+1))
Type "help" for help.

logistics=#
```



Mit `\l` werden über `psql` alle Datenbanken ausgegeben:

```
logistics=# \l
```

Name	Owner	Encoding	Collate	Ctype	ICU Locale	Locale Provider	Access privileges
logistics	user	UTF8	en_US.utf8	en_US.utf8		libc	
postgres	user	UTF8	en_US.utf8	en_US.utf8		libc	
template0	user	UTF8	en_US.utf8	en_US.utf8		libc	=c/user + user=CTc/user
template1	user	UTF8	en_US.utf8	en_US.utf8		libc	=c/user + user=CTc/user

(4 rows)

Schritt 5: in mongodb container

Über `docker exec -it mongodb /bin/bash` wird in den mongodb-container gesprungen. Anschließend wird der mongodb client `mongosh` aufgerufen über:
`mongosh "mongodb://localhost:27017":`

```
ad@ad-ThinkPad-T460s:~/Desktop/1_dma/Aufgaben/logistics_1/databasemodels_logistics_playground/src$ docker exec -it mongodb /bin/bash
root@d8b6ff989280:/# mongosh "mongodb://localhost:27017"
Current Mongosh Log ID: 69f9f3f3a4b72e1ea544ba88
Connecting to:      mongodb://localhost:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.2
Using MongoDB:      7.0.32
Using Mongosh:      2.8.2

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

-----
  The server generated these startup warnings when booting
  2026-05-05T13:07:29.129+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
  2026-05-05T13:07:33.222+00:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
  2026-05-05T13:07:33.222+00:00: Soft rlimits for open file descriptors too low
-----

test> 
```

Über `show databases` Datenbanken anzeigen:

```
test> show databases
admin    40.00 KiB
config   72.00 KiB
local    40.00 KiB
```

Schritt 6: in redis container

Über `docker exec -it redis /bin/bash` wird in den redisdb-container gesprungen.

Anschließend wird der mongodb client redis-cli aufgerufen über:

`redis-cli`

Über `Info keyspace` werden Wert Namensräume angezeigt:

```
ad@ad-ThinkPad-T460s:~$ docker exec -it redis /bin/bash
root@2cd30af5dc93:/data# redis-cli
127.0.0.1:6379> INFO keyspace
# Keyspace
127.0.0.1:6379> 
```

Schritt 7: in neo4j container

Über `docker exec -it neo4j /bin/bash` wird in den neo4j-container gesprungen.

Anschließend wird der mongodb client mongosh aufgerufen über:

`cypher-shell -u neo4j -p password:`

Mit folgendem Befehl werden alle Graphen angezeigt:

`Match (n)-[r]-(m) return n,r,m;`

```
ad@ad-ThinkPad-T460s:~/Desktop/1_dma/Aufgaben/logistics_1/databasemodels_logisti
cs_playground/container$ docker exec -it neo4j /bin/bash
root@2fb1a2f9cb63:/var/lib/neo4j# cypher-shell -u neo4j -p password
Connected to Neo4j using Bolt protocol version 5.8 at neo4j://localhost:7687 as
user neo4j.
Type :help for a list of available commands or :exit to exit the shell.
Note that Cypher queries must end with a semicolon.
neo4j@neo4j> MATCH (n)-[r]-(m)
                RETURN n, r, m;
+-----+
| n | r | m |
+-----+
+-----+

0 rows
ready to start consuming query after 769 ms, results consumed after another 8 ms
```


Schritt 8: in minio container

Über `docker exec -it minio /bin/bash` wird in den mongodb-container gesprungen. Anschließend wird der mongodb client mongosh aufgerufen über:

`Mc alias set myminio http://localhost:9000 admin password`

```
ad@ad-ThinkPad-T460s:~$ docker exec -it minio /bin/bash
bash-5.1# mc alias set myminio http://localhost:9000 admin password
mc: Configuration written to `/tmp/.mc/config.json`. Please update your access credentials.
mc: Successfully created `/tmp/.mc/share`.
mc: Initialized share uploads `/tmp/.mc/share/uploads.json` file.
mc: Initialized share downloads `/tmp/.mc/share/downloads.json` file.
Added `myminio` successfully.
bash-5.1# mc ls myminio
bash-5.1# mc ls myminio/docs
mc: <ERROR> Unable to list folder. Bucket `docs` does not exist.
bash-5.1#
```

Erklärung zu Schritt 9: initialisieren der Datenbanken mit python-Skripts

db: logistics
(in docker-compose.yml)
User: user

Customers ← 10 Einträge

Supplier ← 4 Einträge

Shippers ← 4 Einträge

Orders

OrderDetails

Warehouses ← 4 Einträge

Products ← 10 Einträge

SQL

db: logistics
collection: shipment

insert 1 document:
Order Hamburg → Berlin

mongodb

r.set("orders:counter", 0)

redis

s, w, c namespaces
für supplier, ware-
house, customer

2 Abhängigkeiten

neo4j

bucket: docs

1 Invoice Dokument
1 Delivery Dokument

minio

Schritt 9: initialisieren der Datenbanken mit python-Skript

Ausführung des python Skripts "initialize_db.py"

```
ad@ad-ThinkPad-T460s:~/Desktop/1_dma/Aufgaben/logistics_1/databasemodels_logistics_playground/src$ python3 initialize_db.py
=== INITIALIZING SYSTEM ===
Init PostgreSQL...
Init MongoDB...
Init Redis...
Init Neo4j...
Init MinIO...
=== DONE ===
```

Erklärung zu Schritt 10: Simulation von fullfillment mit Befüllung der Datenbanken

Bitte verschaffen Sie sich ein Überblick, was das Skript `simulate_fullfillment.py` macht (eventuell auch unter Zuhilfenahme von GenAI, um den python code zu verstehen. Der code ist aber auch gut dokumentiert)

Schritt 10: Simulation von fullfillment mit Befüllung der Datenbanken

In das Verzeichnis src springen (z.B. mit cd) und mit python das Skript simulate_fullfillment.py ausführen.

Ausgabe:

```
ad@ad-ThinkPad-T460s:~/Desktop/1_dma/Aufgaben/logistics_1/databasemodels_logistics_playground/src$ python3 simulate_fullfillment.py
Order 1 created
Order 2 created
Order 3 created
Order 4 created
Order 5 created
Order 6 created
Order 7 created
Order 8 created
Order 9 created
Order 10 created
Order 11 created
Order 12 created
Order 13 created
Order 14 created
Order 15 created
Order 16 created
Order 17 created
Order 18 created
Order 19 created
Order 20 created
```

Schritt 11: Ausgabe der SQL-DB Inhalte

In container postgres springen und psql öffnen (siehe Schritt 4 / Folie 13), danach die Befehle der Screenshots eingeben

Ausgabe:

```
logistics=# \c logistics
You are now connected to database "logistics" as user "user".
logistics=# \dt
          List of relations
Schema |      Name      | Type  | Owner
-----+-----+-----+-----
public | customers      | table | user
public | orderdetails   | table | user
public | orders         | table | user
public | products       | table | user
public | shippers       | table | user
public | suppliers      | table | user
public | warehouses     | table | user
(7 rows)
```

```
logistics=# select * from customers;
customerid | customername | city    | country
-----+-----+-----+-----
          1 | TechCorp    | Munich  | Germany
          2 | RetailHub   | Cologne | Germany
          3 | Foodies     | Hamburg | Germany
          4 | GreenMarket | Berlin  | Germany
          5 | UrbanShop   | Frankfurt | Germany
          6 | Nordic AB   | Stockholm | Sweden
          7 | Paris Store | Paris   | France
          8 | London Goods | London  | UK
          9 | Iberia Trade | Madrid  | Spain
         10 | Roma Market | Rome    | Italy
(10 rows)
```

Schritt 11: Ausgabe der SQL-DB Inhalte

```
logistics=# select * from orders;
```

orderid	customerid	warehouseid	orderdate	shipperid
1	6	4	2026-05-05 21:57:00.951845	4
2	8	3	2026-05-05 21:57:01.315733	2
3	1	3	2026-05-05 21:57:01.506419	3
4	5	4	2026-05-05 21:57:01.707638	1
5	6	4	2026-05-05 21:57:01.840133	2
6	9	1	2026-05-05 21:57:02.019412	2
7	2	1	2026-05-05 21:57:02.257526	1
8	7	4	2026-05-05 21:57:02.461019	3
9	1	3	2026-05-05 21:57:02.72112	1
10	6	4	2026-05-05 21:57:02.872335	1
11	8	4	2026-05-05 21:57:03.027199	4
12	4	3	2026-05-05 21:57:03.249307	3
13	6	4	2026-05-05 21:57:03.486148	3
14	2	2	2026-05-05 21:57:03.77096	2
15	2	3	2026-05-05 21:57:03.997361	2
16	8	3	2026-05-05 21:57:04.279633	4
17	7	1	2026-05-05 21:57:04.473834	1
18	4	4	2026-05-05 21:57:04.776977	4
19	5	1	2026-05-05 21:57:05.003276	2
20	5	2	2026-05-05 21:57:05.26499	3

```
(20 rows)
```

Schritt 12: Ausgabe der mongo-DB Inhalte

In container mongoddb springen und den mongoddb client öffnen (siehe Schritt 5 / Folie 14), danach die Befehle der Screenshots eingeben

Ausgabe:

```
test> show databases
admin      40.00 KiB
config     108.00 KiB
local      40.00 KiB
logistics  108.00 KiB
```

```
test> use logistics
switched to db logistics
```

```
logistics> show collections
shipments
[
  {
    _id: ObjectId('69fa66ee2593b2cf68f64cdf'),
    orderId: 1,
    route: [ 'Hamburg', 'Berlin' ],
    events: [
      { status: 'created', timestamp: ISODate('2026-05-05T21:53:50.364Z') }
    ],
    createdAt: ISODate('2026-05-05T21:53:50.364Z')
  },
  {
    _id: ObjectId('69fa67ac254205acfb8674a7'),
    orderId: 1,
    customerId: 6,
    warehouseId: 4,
    status: 'created',
    events: [
      { status: 'created', timestamp: ISODate('2026-05-05T21:57:00.958Z') }
    ],
    createdAt: ISODate('2026-05-05T21:57:00.958Z')
  },
]
```


Schritt 12: Ausgabe der redis-DB Inhalte

In container redis springen und redis client öffnen (siehe Schritt 6 / Folie 15), danach die Befehle der Screenshots eingeben

Ausgabe:

```
root@2cd30af5dc93:/data# redis-cli
127.0.0.1:6379> KEYS *
```

- 1) "order:20:meta"
- 2) "order:9:timeline"
- 3) "order:10:status"
- 4) "order:16:meta"
- 5) "order:15:meta"
- 6) "order:7:timeline"
- 7) "order:14:timeline"
- 8) "order:15:timeline"
- 9) "order:5:status"
- 10) "order:4:meta"
- 11) "order:6:timeline"
- 12) "order:10:timeline"
- 13) "order:17:meta"
- 14) "order:7:meta"
- 15) "order:9:status"
- 16) "order:20:status"

Schritt 12: Ausgabe der neo4j-DB Inhalte

In container neo4j springen und neo4j client öffnen (siehe Schritt 7 / Folie 16), danach die Befehle der Screenshots eingeben

Ausgabe:

```
ready to start consuming query after 769 ms, results consumed after another 8 ms
neo4j@neo4j>

Bye!
root@2fb1a2f9cb63:/var/lib/neo4j# cypher-shell -u neo4j -p password
Connected to Neo4j using Bolt protocol version 5.8 at neo4j://localhost:7687 as
user neo4j.
Type :help for a list of available commands or :exit to exit the shell.
Note that Cypher queries must end with a semicolon.
neo4j@neo4j> 
```

Schritt 12: Ausgabe der neo4j-DB Inhalte

```
neo4j@neo4j> MATCH (n)-[r]->(m)
               RETURN n, r, m;
```

n	r	m
(:Customer {id: 2})	[[:PLACED]]	(:Order {id: 15})
(:Warehouse {id: 3})	[[:FULFILLS]]	(:Order {id: 15})
(:Supplier {name: "Global Foods", id: 1})	[[:SUPPLIES]]	(:Warehouse {city: "Hamburg", id: 1})
(:Customer {name: "TechCorp", id: 101})	[[:ORDERS]]	(:Warehouse {city: "Hamburg", id: 1})
(:Customer {id: 6})	[[:PLACED]]	(:Order {id: 1})
(:Order {id: 15})	[[:CONTAINS]]	(:Product {id: 7})
(:Order {id: 15})	[[:CONTAINS]]	(:Product {id: 9})
(:Warehouse {id: 4})	[[:FULFILLS]]	(:Order {id: 1})
(:Order {id: 1})	[[:CONTAINS]]	(:Product {id: 9})
(:Product {id: 9})	[[:SUPPLIED_BY]]	(:Supplier {name: "Global Foods", id: 1})
(:Customer {id: 8})	[[:PLACED]]	(:Order {id: 16})
(:Warehouse {id: 3})	[[:FULFILLS]]	(:Order {id: 16})
(:Supplier {name: "Global Foods", id: 1})	[[:SUPPLIES]]	(:Warehouse {id: 4})
(:Order {id: 1})	[[:CONTAINS]]	(:Product {id: 6})
(:Product {id: 6})	[[:SUPPLIED_BY]]	(:Supplier {id: 3})
(:Order {id: 16})	[[:CONTAINS]]	(:Product {id: 9})
(:Customer {id: 7})	[[:PLACED]]	(:Order {id: 17})
(:Supplier {id: 3})	[[:SUPPLIES]]	(:Warehouse {id: 4})
(:Customer {id: 8})	[[:PLACED]]	(:Order {id: 2})
(:Warehouse {id: 3})	[[:FULFILLS]]	(:Order {id: 2})

Schritt 12: Ausgabe der minio-DB Inhalte

In den container minio springen und den minio client öffnen (siehe Schritt 8 / Folie 17),
danach die Befehle der Screenshots eingeben

Ausgabe:

```
ad@ad-ThinkPad-T460s:~$ docker exec -it minio /bin/bash
bash-5.1# mc alias set myminio http://localhost:9000 admin password
Added 'myminio' successfully.
bash-5.1# mc ls myminio/docs
[2026-05-05 21:53:50 UTC] 21B STANDARD delivery_note_1.pdf
[2026-05-05 21:53:50 UTC] 15B STANDARD invoice_1.pdf
[2026-05-05 23:10:36 UTC] 0B orders/
bash-5.1# mc ls myminio
[2026-05-05 21:53:50 UTC] 0B docs/
bash-5.1# mc ls myminio/docs
[2026-05-05 21:53:50 UTC] 21B STANDARD delivery_note_1.pdf
[2026-05-05 21:53:50 UTC] 15B STANDARD invoice_1.pdf
[2026-05-05 23:11:16 UTC] 0B orders/
bash-5.1# mc ls myminio/docs/orders
[2026-05-05 23:11:21 UTC] 0B 1/
[2026-05-05 23:11:21 UTC] 0B 10/
[2026-05-05 23:11:21 UTC] 0B 11/
[2026-05-05 23:11:21 UTC] 0B 12/
[2026-05-05 23:11:21 UTC] 0B 13/
[2026-05-05 23:11:21 UTC] 0B 14/
[2026-05-05 23:11:21 UTC] 0B 15/
[2026-05-05 23:11:21 UTC] 0B 16/
[2026-05-05 23:11:21 UTC] 0B 17/
[2026-05-05 23:11:21 UTC] 0B 18/
[2026-05-05 23:11:21 UTC] 0B 19/
[2026-05-05 23:11:21 UTC] 0B 2/
[2026-05-05 23:11:21 UTC] 0B 20/
[2026-05-05 23:11:21 UTC] 0B 3/
[2026-05-05 23:11:21 UTC] 0B 4/
[2026-05-05 23:11:21 UTC] 0B 5/
[2026-05-05 23:11:21 UTC] 0B 6/
[2026-05-05 23:11:21 UTC] 0B 7/
[2026-05-05 23:11:21 UTC] 0B 8/
[2026-05-05 23:11:21 UTC] 0B 9/
bash-5.1# mc ls myminio/docs/orders/1
[2026-05-05 21:57:01 UTC] 71B STANDARD delivery.txt
[2026-05-05 21:57:01 UTC] 104B STANDARD invoice.txt
bash-5.1# mc ls myminio/docs/orders/2
[2026-05-05 21:57:01 UTC] 62B STANDARD delivery.txt
[2026-05-05 21:57:01 UTC] 92B STANDARD invoice.txt
bash-5.1# mc cat myminio/docs/orders/2/delivery.txt
DELIVERY 2
Warehouse: WH_Munich

Ikura -> 9
```

Schritt 13: Datenbanken aufräumen

Führen Sie die Skripte `python3 cleanup_initialized_db.py` und `cleanup_simulated_data.py` aus, um die generierten Datenbankinhalte zu löschen:

```
ad@ad-ThinkPad-T460s:~/Desktop/1_dma/Aufgaben/logistics_1/databasemodels_logistics_playground/src$ python3 cleanup_initialized_db.py
=== RESETTING SYSTEM ===
Reset PostgreSQL...
Reset MongoDB...
Reset Redis...
Reset Neo4j...
Reset MinIO...
=== RESET DONE ===
ad@ad-ThinkPad-T460s:~/Desktop/1_dma/Aufgaben/logistics_1/databasemodels_logistics_playground/src$ python3 cleanup_simulated_data.py
=== CLEANING SIMULATED DATA ===
Cleaning PostgreSQL (orders only)...
Cleaning MongoDB shipments...
Cleaning Redis simulation keys...
Cleaning Neo4j simulation graph...
Cleaning MinIO order documents...
=== CLEANUP DONE ===
```

Schritt 14: IT-Infrastruktur lokal entfernen

Sie können die Docker container final (spätestens am Ende der Vorlesung, sonst auch jeder Zeit, nur dann müssen Sie wieder alle Schritte von vorne beginnen) über den Befehl:

```
docker compose down -v --rmi all -remove-orphans
```

entfernen.